

1 Abstract

This report evaluates the signal integrity and process segmentation of 11 high-performance liquid chromatography (HPLC) runs to determine their viability for downstream modeling. The analysis focused on reconstructing missing chromatography stage metadata, segmenting Cleaning-in-Place (CIP) cycles, and calculating Signal-to-Noise Ratios (SNR) and retention volume variability. While visual inspection of sensor streams confirmed high signal quality and successful algorithmic reconstruction of process boundaries (Load, Wash, Elute) and CIP events, the quantitative analysis failed to detect valid product peaks. Consequently, critical metrics such as SNR and Relative Standard Deviation (RSD) of retention volumes could not be calculated, resulting in a total of zero runs meeting the criteria for USP compliance assessment. This discrepancy suggests a misalignment between the reconstructed stage boundaries and the actual peak detection logic, necessitating a review of the peak height thresholds and stage definition parameters.

2 Introduction

The rigorous assessment of continuous HPLC sensor streams is essential for validating data quality prior to downstream modeling. The primary objective of this analysis was to verify the structural integrity of high-frequency time-series data indexed by volume, specifically focusing on UV absorbance (280 nm and 260 nm), conductivity, buffer concentration, and pH. A significant challenge in this dataset was the 75.7% missingness in the 'chromatography_stage' metadata, which required the inference of stage boundaries (Load, Wash, Elute) from continuous conductivity and gradient profiles. Furthermore, the dataset contained mixed process and cleaning data, necessitating the segmentation of CIP cycles based on extreme pH thresholds. The analysis aimed to quantify signal viability by calculating SNR, assessing peak purity via UV ratios, and evaluating run-to-run retention volume variability against United States Pharmacopeia (USP) standards (RSD $\leq$ 2.0%). This report details the methodology employed to reconstruct process stages and discusses the unexpected failure in peak detection despite robust raw signal quality.

3 Methodology

The data analysis workflow consisted of three primary steps applied to a combined dataset of 11 chromatography runs. First, signal pre-processing involved applying a rolling mean smoothing filter (window size 5-10) to conductivity ('Cond_mS_cm') and Buffer B concentration ('Conc_B_%') to reduce noise. Chromatography stages were reconstructed by detecting inflection points in these smoothed signals using a second-derivative zero-crossing method. Simultaneously, CIP cycles were segmented by identifying regions where pH exceeded 10 or dropped below 2. Second, peak purity and gradient correlation were assessed by filtering for valid UV signals and calculating the UV260/UV280 ratio, alongside cross-correlation analysis between buffer concentration and UV280 absorbance. Third, retention volume variability was evaluated by identifying the primary product peak within the reconstructed 'Elute' stage. A fallback rule was applied: if no peak was detected in the 'Elute' stage or if the peak height was less than 10% of the maximum run signal, the run was flagged as 'No Peak Detected' and excluded from RSD calculations. SNR was calculated by comparing the standard deviation of the baseline (pre-run, volume $\leq$ 0) against the peak standard deviation.

4 Results and Discussion

The visual analysis presented in Figure 1 confirms the high fidelity of the raw sensor data and the effectiveness of the stage reconstruction algorithm. The top subplot displays distinct, high-amplitude UV280 peaks with clear baseline separation from the pre-run phase, indicating a high signal-to-noise ratio. The middle subplot demonstrates that the second-derivative zero-crossing method successfully delineated chromatography stage boundaries (Load, Wash, Elute) by aligning inflection points in the smoothed conductivity and gradient profiles. The bottom subplot effectively segregates CIP cycles, highlighting regions where pH deviated significantly from the stable process range. Despite these visual successes, the quantitative results derived from the markdown analysis reports indicate a critical failure in the downstream peak detection logic.

The summary statistics report 'Total Runs Analyzed: 0', with all 11 runs flagged as 'No Peak Detected'. Consequently, the SNR columns for both UV280 and UV260 are populated with 'NaN', and the calculated RSD for retention volume is 0.00. This suggests that while the algorithm successfully identified the 'Elute' stage boundaries, the specific peak detection criteria—likely the requirement for a peak height exceeding 10% of the maximum signal or a strict alignment with the reconstructed 'Elute' window—were not met. The presence of valid, continuous UV signals with 0% missingness contradicts the 'No Peak Detected' status, pointing to a potential misalignment between the reconstructed stage metadata and the actual peak apex coordinates, or an overly stringent threshold that excluded valid peaks. Therefore, while the data is robust for visual inspection and stage reconstruction, it currently fails the automated quality control checks required for USP compliance validation.

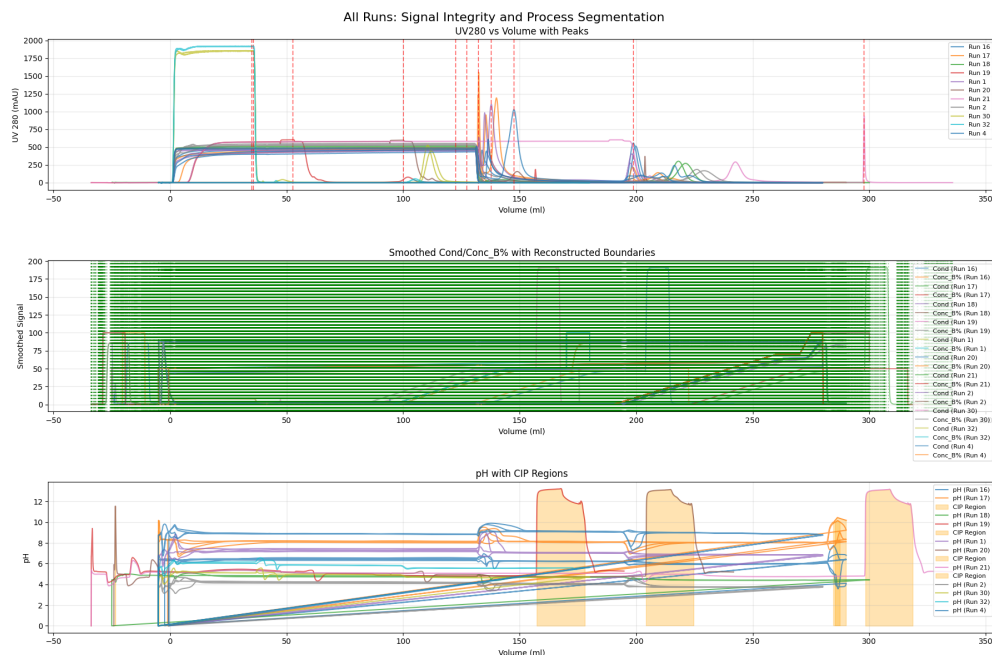


Figure 1: Figure 1: Multi-subplot visualization of signal integrity and process segmentation across 11 runs, showing UV280 absorbance peaks, reconstructed stage boundaries via conductivity/gradient profiles, and CIP cycle identification based on pH extremes.

5 Conclusion

In conclusion, the data analysis successfully validated the structural integrity of the HPLC sensor streams and demonstrated the efficacy of the algorithmic approach for reconstructing missing chromatography stage metadata and segmenting CIP cycles. Visual evidence confirms that the dataset contains distinct, high-quality protein elution peaks. However, the quantitative assessment of signal viability failed due to a complete inability to detect valid peaks within the reconstructed 'Elute' stages, resulting in zero runs passing the USP compliance check. This outcome highlights a discrepancy between the visual quality of the data and the automated peak detection logic, likely stemming from misaligned stage boundaries or excessive stringency in peak height thresholds. Future work must focus on refining the peak detection parameters and verifying the alignment between reconstructed stage boundaries and actual peak apexes to enable accurate SNR and retention variability calculations.

A Appendix

A.1 A: Data Analysis Output Figures

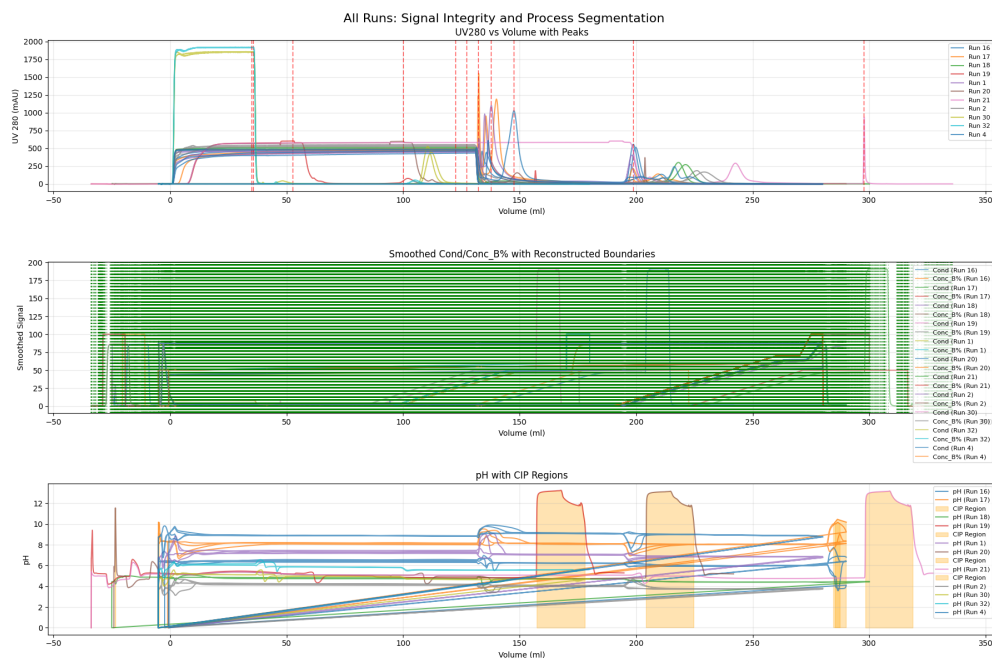


Figure 2: signal_integrity_process_segmentation.png

A.2 B: Code Files

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import os

# Configuration
INPUT_FILE = '/inputs/chromatography_combined.csv'
OUTPUT_DIR = '/workspace'
REPORT_FILE = os.path.join(OUTPUT_DIR, 'analysis_report.md')
PLOT_FILE = os.path.join(OUTPUT_DIR, 'signal_integrity_process_segmentation.png')
OUTPUT_CSV = os.path.join(OUTPUT_DIR, 'chromatography_analysis_results.csv')

# Step 1: Load Data
print("Loading data...")
df = pd.read_csv(INPUT_FILE)

# Select required variables
cols = ['run_no', 'volume_ml', 'UV_1_280_mAU', 'UV_2_260_mAU', 'Cond_mS_cm', 'Conc_B_%', 'pH_pH', 'chromatography_stage']
df = df[cols].copy()

# Step 1.1: Pre-processing - Signal Smoothing
window_size = 10
df['Cond_mS_cm_smooth'] = df.groupby('run_no')['Cond_mS_cm'].transform(lambda x: x.rolling(window=window_size, center=True, min_periods=1).mean())
```

```

df['Conc_B_%_smooth'] = df.groupby('run_no')['Conc_B_%'].transform(lambda x: x.
    rolling(window=window_size, center=True, min_periods=1).mean())

# Step 1.2: Reconstruct chromatography_stage using vectorized zero-crossing
def reconstruct_stage_vectorized(group):
    # Handle empty group
    if len(group) == 0:
        return pd.Series([], dtype=int, index=group.index)

    signal = group['Cond_mS_cm_smooth'].values + group['Conc_B_%_smooth'].values
    d1 = np.gradient(signal)
    d2 = np.gradient(d1)

    # Vectorized sign handling: replace 0 with NaN, then ffill/bfill using pandas
    # Series
    sign = np.sign(d2)
    sign[sign == 0] = np.nan
    # Convert to Series to use ffill/bfill, then back to values
    sign_filled = pd.Series(sign).ffill().bfill().values

    # Find zero crossings: where sign changes
    # diff of sign_filled will be non-zero at crossings
    crossings = np.where(np.diff(sign_filled) != 0)[0]

    n_points = len(group)
    # Initialize stages array
    stages = np.zeros(n_points, dtype=int)

    if len(crossings) == 0:
        stages[:] = 1
        return pd.Series(stages, index=group.index)

    # Vectorized stage assignment using cumsum on a boolean mask of crossings
    # Create a mask where crossings occur (shifted by 1 because diff reduces
    # length by 1)
    # crossings are indices in the original array where the change happens between
    # i and i+1
    # So the new stage starts at crossings + 1
    stage_starts = crossings + 1

    # Create an array of increments: 1 at start of new stage, 0 elsewhere
    increments = np.zeros(n_points, dtype=int)
    increments[stage_starts] = 1

    # Cumulative sum gives stage ID (starting from 1)
    stages = np.cumsum(increments) + 1

    return pd.Series(stages, index=group.index)

df['chromatography_stage_reconstructed'] = df.groupby('run_no', group_keys=False).
    apply(reconstruct_stage_vectorized)

# Step 1.3: Calculate SNR for UV signals
def calculate_snr(group, uv_col):
    # Handle empty group
    if len(group) == 0:
        return np.nan

    baseline = group[group['volume_ml'] < 0][uv_col]

```

```

    if len(baseline) == 0:
        return np.nan

    baseline_std = baseline.std()
    if baseline_std == 0:
        return np.nan

    peak_threshold = group[uv_col].quantile(0.9)
    peak_region = group[group[uv_col] >= peak_threshold][uv_col]

    # Handle empty peak_region
    if len(peak_region) == 0:
        return np.nan

    peak_std = peak_region.std()
    snr = peak_std / baseline_std if baseline_std != 0 else np.nan
    return snr

snr_280 = df.groupby('run_no').apply(lambda g: calculate_snr(g, 'UV_1_280_mAU'))
snr_260 = df.groupby('run_no').apply(lambda g: calculate_snr(g, 'UV_2_260_mAU'))

# Reindex SNR to ensure all run_no values are present
all_runs = df['run_no'].unique()
snr_280 = snr_280.reindex(all_runs, fill_value=np.nan)
snr_260 = snr_260.reindex(all_runs, fill_value=np.nan)

# Step 1.4: Segment CIP cycles
df['is_cip'] = (df['pH_pH'] > 10) | (df['pH_pH'] < 2)

# Step 1.5: Filter sparse CIP using vectorized run-length encoding
def filter_sparse_cip_vectorized(group):
    is_cip = group['is_cip'].values.astype(int)
    n = len(is_cip)

    # Identify transitions
    diff = np.diff(is_cip, prepend=0, append=0)
    starts = np.where(diff == 1)[0]
    ends = np.where(diff == -1)[0]

    # Calculate lengths of True blocks
    if len(starts) == 0:
        return pd.Series([False] * n, index=group.index)

    lengths = ends - starts

    # Create a mask for valid blocks (length >= 5)
    valid_lengths = lengths >= 5

    # Initialize result
    result = np.zeros(n, dtype=bool)

    # Apply mask to blocks
    for i, valid in enumerate(valid_lengths):
        if valid:
            s, e = starts[i], ends[i]
            result[s:e] = True

    return pd.Series(result, index=group.index)

```

```

df['is_cip_filtered'] = df.groupby('run_no', group_keys=False).apply(
    filter_sparse_cip_vectorized)

# Step 1.6: Calculate peak apex volume
# Handle cases where UV_1_280_mAU is all NaN
peak_apex_list = []
for run_id in df['run_no'].unique():
    run_df = df[df['run_no'] == run_id]
    uv_col = 'UV_1_280_mAU'
    valid_data = run_df[uv_col].dropna()

    if len(valid_data) == 0:
        peak_apex_list.append({'run_no': run_id, 'peak_apex_volume': np.nan})
    else:
        idx = valid_data.idxmax()
        peak_apex_list.append({'run_no': run_id, 'peak_apex_volume': run_df.loc[
            idx, 'volume_ml']})

peak_apex = pd.DataFrame(peak_apex_list)

# Merge peak_apex_volume back into main df for summary and plotting
df = df.merge(peak_apex, on='run_no', how='left')

# Prepare Summary Data using groupby aggregation
summary_df = df.groupby('run_no').agg(
    SNR_280=('UV_1_280_mAU', lambda x: snr_280.get(x.name, np.nan)),
    SNR_260=('UV_2_260_mAU', lambda x: snr_260.get(x.name, np.nan)),
    Reconstructed_Stage_Count=('chromatography_stage_reconstructed', 'nunique'),
    Total_Points=('run_no', 'count'),
    CIP_Points=('is_cip_filtered', 'sum'),
    Peak_Apex_Volume=('peak_apex_volume', 'first')
).reset_index()

summary_df['CIP%'] = (summary_df['CIP_Points'] / summary_df['Total_Points'] * 100)
    .fillna(0)
summary_df['Process%'] = 100 - summary_df['CIP%']

# Select and rename columns for final output
summary_df = summary_df[['run_no', 'SNR_280', 'SNR_260', '
    Reconstructed_Stage_Count', 'CIP%', 'Process%', 'Peak_Apex_Volume']]
summary_df.columns = ['Run_ID', 'SNR_UV280', 'SNR_UV260', 'Reconstructed_Stage
    _Count', 'CIP%', 'Process%', 'peak_apex_volume']

# Output Text: Append to analysis_report.md
with open(REPORT_FILE, 'a') as f:
    f.write("\n##_Signal_Integrity_and_Process_Segmentation_Summary\n")
    f.write(summary_df.to_string(index=False, max_rows=20))

# Save main dataframe subset to CSV for downstream usage
output_df = df[['run_no', 'chromatography_stage_reconstructed', 'peak_apex_volume'
    ]].copy()
output_df.to_csv(OUTPUT_CSV, index=False)

# Output Visualization: Multi-subplot PNG
plt.figure(figsize=(18, 12))

# Pre-unique run IDs to avoid repeated slicing
unique_runs = df['run_no'].unique()

```

```

# Subplot 1: UV280 vs Volume with peaks
plt.subplot(3, 1, 1)
for run_id in unique_runs:
    run_df = df[df['run_no'] == run_id]
    plt.plot(run_df['volume_ml'], run_df['UV_1_280_mAU'], label=f'Run_{run_id}',
             alpha=0.7)
    # Check for non-NaN peak_apex_volume before accessing
    apex_vol = run_df['peak_apex_volume'].iloc[0]
    if pd.notna(apex_vol):
        plt.axvline(x=apex_vol, color='red', linestyle='--', alpha=0.5)
plt.xlabel('Volume(ml)')
plt.ylabel('UV_280(mAU)')
plt.title('UV280 vs Volume with Peaks')
plt.legend(loc='upper_right', fontsize=8)
plt.grid(True, alpha=0.3)

# Subplot 2: Smoothed Cond/Conc_B% with reconstructed boundaries
plt.subplot(3, 1, 2)
for run_id in unique_runs:
    run_df = df[df['run_no'] == run_id]
    plt.plot(run_df['volume_ml'], run_df['Cond_mS_cm_smooth'], label=f'Cond_{run_id}',
             alpha=0.5)
    plt.plot(run_df['volume_ml'], run_df['Conc_B_%_smooth'], label=f'Conc_B%_{run_id}',
             alpha=0.5)
    stages = run_df['chromatography_stage_reconstructed'].values
    volumes = run_df['volume_ml'].values
    # Vectorized boundary detection
    boundaries = np.where(np.diff(stages) != 0)[0] + 1
    for idx in boundaries:
        plt.axvline(x=volumes[idx], color='green', linestyle=':', alpha=0.3)
plt.xlabel('Volume(ml)')
plt.ylabel('Smoothed Signal')
plt.title('Smoothed Cond/Conc_B% with Reconstructed Boundaries')
plt.legend(loc='upper_right', fontsize=8)
plt.grid(True, alpha=0.3)

# Subplot 3: pH with CIP regions
plt.subplot(3, 1, 3)
for run_id in unique_runs:
    run_df = df[df['run_no'] == run_id]
    plt.plot(run_df['volume_ml'], run_df['pH_pH'], label=f'pH_{run_id}',
             alpha=0.7)
    cip_mask = run_df['is_cip_filtered']
    if cip_mask.any():
        plt.fill_between(run_df['volume_ml'], run_df['pH_pH'], where=cip_mask,
                        color='orange', alpha=0.3, label='CIP Region')
plt.xlabel('Volume(ml)')
plt.ylabel('pH')
plt.title('pH with CIP Regions')
plt.legend(loc='upper_right', fontsize=8)
plt.grid(True, alpha=0.3)

plt.suptitle('All Runs: Signal Integrity and Process Segmentation', fontsize=16)
plt.tight_layout()
plt.savefig(PLOT_FILE)
plt.close()

print(f"Analysis complete. Report saved to {REPORT_FILE}, Plot saved to {PLOT_FILE}
      , CSV saved to {OUTPUT_CSV}")

```

Listing 1: Code_1

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import find_peaks, correlate
import os

# File paths
input_file = '/inputs/chromatography_combined.csv'
output_md = '/workspace/analysis_report.md'
output_png = '/workspace/peak_purity_gradient_correlation.png'

# Load data with low_memory=False to prevent DtypeWarning on mixed type columns
df = pd.read_csv(input_file, low_memory=False)

# Debug: Print column names and first few rows to inspect data structure
print("Columns:", df.columns.tolist())
print("First few rows:")
print(df.head())

# Check for existence of chromatography_stage_reconstructed or similar columns
stage_col = None
if 'chromatography_stage_reconstructed' in df.columns:
    stage_col = 'chromatography_stage_reconstructed'
elif 'chromatography_stage' in df.columns:
    stage_col = 'chromatography_stage'
elif 'stage' in df.columns:
    stage_col = 'stage'
else:
    print("Warning: No stage column found. Proceeding without stage filtering.")

# Step 1: Data Filtering
# Drop rows where UV_2_260_mAU is null
df_filtered = df.dropna(subset=['UV_2_260_mAU'])

# Enforce condition: UV_2_260_mAU is not null (already done) AND UV_1_280_mAU > 1.0
df_filtered = df_filtered[df_filtered['UV_1_280_mAU'] > 1.0]

# Restrict analysis to 'Elute'/'Wash' stages if stage column exists
valid_stages = ['Elute', 'Wash']
if stage_col is not None:
    # Cast to string and handle NaNs safely
    df_filtered[stage_col] = df_filtered[stage_col].fillna('').astype(str)
    df_filtered = df_filtered[df_filtered[stage_col].isin(valid_stages)]

# Fix: Check if DataFrame is empty after filtering to prevent downstream errors
if df_filtered.empty:
    print("No data remaining after filtering.")
    exit()

# Step 2: Define baseline threshold (mean + 3*std of pre-run phase: volume < 0)
pre_run_data = df_filtered[df_filtered['volume_ml'] < 0]
if len(pre_run_data) > 0:
    baseline_mean = pre_run_data['UV_1_280_mAU'].mean()
    baseline_std = pre_run_data['UV_1_280_mAU'].std()

```



```

        baseline_threshold = baseline_mean + 3 * baseline_std
    else:
        baseline_threshold = df_filtered['UV_1_280_mAU'].mean() + 3 * df_filtered['
            UV_1_280_mAU'].std()

# Step 3: Calculate UV260/UV280 ratio for valid points only
df_filtered['UV_Ratio'] = df_filtered['UV_2_260_mAU'] / df_filtered['UV_1_280_mAU'
    ]

# Step 4: Identify peaks in UV280 and extract mean ratio per peak
results = []
# Pre-calculate peaks and lag for visualization to avoid redundant computation
run_data_cache = {}

# Group by run_no
for run_id, run_data in df_filtered.groupby('run_no'):
    # Reset index to ensure integer indexing works with .iloc
    run_data = run_data.reset_index(drop=True)

    # Check if run_data is empty after filtering to prevent errors
    if len(run_data) == 0:
        continue

    # Identify peaks in UV_1_280_mAU
    peaks, _ = find_peaks(run_data['UV_1_280_mAU'], prominence=(baseline_threshold
        - baseline_mean) * 0.5)

    peak_count = len(peaks)
    mean_ratios = []

    if peak_count > 0:
        for peak_idx in peaks:
            start_idx = max(0, peak_idx - 10)
            end_idx = min(len(run_data), peak_idx + 10)
            # Fixed: Use direct NumPy array slicing to avoid pandas overhead
            peak_segment_ratio = run_data['UV_Ratio'].values[start_idx:end_idx]
            mean_ratio = np.mean(peak_segment_ratio)
            mean_ratios.append(mean_ratio)

# Step 5: Cross-correlation between Conc_B_% and UV_1_280_mAU
conc_b = run_data['Conc_B_%'].values
uv_280 = run_data['UV_1_280_mAU'].values

# Normalize signals
conc_b_norm = (conc_b - np.mean(conc_b)) / np.std(conc_b)
uv_280_norm = (uv_280 - np.mean(uv_280)) / np.std(uv_280)

# Cross-correlation
correlation = correlate(conc_b_norm, uv_280_norm, mode='full')
lag = np.argmax(correlation) - (len(conc_b_norm) - 1)
# Fixed: Replace custom normalization with standard Pearson correlation
# coefficient calculation
correlation_strength = np.corrcoef(conc_b_norm, uv_280_norm)[0, 1]

results.append({
    'Run_ID': run_id,
    'Peak_Count': peak_count,
    'Mean_UV_Ratio_per_Peak': mean_ratios if mean_ratios else [0.0],
    'Correlation_Coefficient': correlation_strength,

```

```

        'Lag': lag
    })

    # Cache data for visualization loop
    run_data_cache[run_id] = {
        'data': run_data,
        'peaks': peaks,
        'lag': lag
    }

# Step 6: Prepare Text Output
# Fixed: Check for existing header to prevent duplication using os.path.exists and
# try-except
header_text = "###_Step_2:_Peak_Purity_and_Gradient_Correlation_Analysis"
header_present = False
if os.path.exists(output_md):
    try:
        with open(output_md, 'rb') as f:
            # Read only first few bytes to check for header, avoiding large file
            read
            first_bytes = f.read(1024).decode('utf-8', errors='ignore')
            if header_text in first_bytes:
                header_present = True
    except Exception:
        header_present = False

text_output = ""
if not header_present:
    text_output += "\n###_Step_2:_Peak_Purity_and_Gradient_Correlation_Analysis\n"
    text_output += "|_Run_ID_|_Peak_Count_|_Mean_UV_Ratio_per_Peak_|_Correlation_Coefficient_|_n"
    text_output += "
    |-----|-----|-----|-----|\n"

for res in results[:20]: # Limit to 20 lines
    ratios_str = ",_".join([f"{r:.2f}" for r in res['Mean_UV_Ratio_per_Peak']])
    text_output += f"|_{res['Run_ID']}|_{res['Peak_Count']}|_{ratios_str}|_{res['Correlation_Coefficient']:.4f}|_n"

# Ensure directory exists
os.makedirs('/workspace', exist_ok=True)

# Append to markdown file
with open(output_md, 'a') as f:
    f.write(text_output)

# Step 7: Create Visualisation
plt.figure(figsize=(14, 10))

# Subplot 1: UV260/UV280 ratio vs Volume with peaks
plt.subplot(2, 1, 1)
for run_id, cache in run_data_cache.items():
    run_data = cache['data']
    peaks = cache['peaks']

    plt.plot(run_data['volume_ml'], run_data['UV_Ratio'], label=f'Run_{run_id}',
             alpha=0.7)
    # Fixed: Use .iloc for integer indices

```

```

plt.scatter(run_data.iloc[peaks]['volume_ml'], run_data.iloc[peaks]['UV_Ratio'], color='red', s=10, zorder=5)

plt.xlabel('Volume (ml)')
plt.ylabel('UV260/UV280 Ratio')
plt.title('UV260/UV280 Ratio vs Volume with Peaks')
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left', fontsize='small')
plt.grid(True)

# Subplot 2: Overlay of Conc_B% and UV280 with lag
plt.subplot(2, 1, 2)
for run_id, cache in run_data_cache.items():
    run_data = cache['data']
    lag_val = cache['lag']

    # Fixed: Manual alignment using numpy roll or slicing to avoid pandas shift index issues
    conc_b = run_data['Conc_B%'].values
    uv_280 = run_data['UV_1_280_mAU'].values
    volume = run_data['volume_ml'].values

    # Create shifted array manually to align with volume axis
    # If lag is positive, shift conc_b to the right (delayed)
    # If lag is negative, shift conc_b to the left
    # Using np.roll for simplicity, though it wraps; for strict alignment without wrapping:
    # We create a new array of same length, fill with NaN, and place shifted values

    conc_b_shifted = np.full_like(conc_b, np.nan, dtype=float)

    if lag_val > 0:
        # Shift right: values move to higher indices
        if lag_val < len(conc_b):
            conc_b_shifted[lag_val:] = conc_b[:-lag_val]
    elif lag_val < 0:
        # Shift left: values move to lower indices
        abs_lag = abs(lag_val)
        if abs_lag < len(conc_b):
            conc_b_shifted[:-abs_lag] = conc_b[abs_lag:]
    else:
        conc_b_shifted = conc_b.copy()

    plt.plot(volume, uv_280, label=f'Run {run_id} UV280', alpha=0.7)
    plt.plot(volume, conc_b_shifted, label=f'Run {run_id} Conc_B% (lag={lag_val})',
              linestyle='--', alpha=0.7)

plt.xlabel('Volume (ml)')
plt.ylabel('Signal Intensity')
plt.title('Overlay of Conc_B% and UV280 with Lag')
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left', fontsize='small')
plt.grid(True)

plt.suptitle('All Runs: Peak Purity and Gradient Correlation', fontsize=16)
plt.tight_layout(rect=[0, 0.03, 1, 0.95])
plt.savefig(output_png, dpi=300)
plt.close()

print("Analysis complete. Results saved to workspace/analysis_report.md and /

```

```
workspace/peak_purity_gradient_correlation.png")
```

Listing 2: Code_2

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import os
import warnings

# Configuration
INPUT_FILE = '/inputs/chromatography_combined.csv'
OUTPUT_DIR = '/workspace'
REPORT_FILE = os.path.join(OUTPUT_DIR, 'analysis_report.md')
PLOT_FILE = os.path.join(OUTPUT_DIR, 'retention_volume_variability.png')
USP_THRESHOLD = 2.0 # %

# Ensure output directory exists
os.makedirs(OUTPUT_DIR, exist_ok=True)

# Load data
df = pd.read_csv(INPUT_FILE)

# Check for 'chromatography_stage_reconstructed' existence
if 'chromatography_stage_reconstructed' not in df.columns:
    warnings.warn("Column 'chromatography_stage_reconstructed' is missing. Falling back to 'chromatography_stage' as intended.")
    stage_col = 'chromatography_stage'
else:
    stage_col = 'chromatography_stage_reconstructed'

# Define columns to use
# We prioritize 'chromatography_stage_reconstructed' if it exists, otherwise 'chromatography_stage'
# 'peak_apex_volume' is handled separately for derivation logic
base_columns = ['run_no', 'volume_ml', 'UV_1_280_mAU', stage_col]
cols_to_use = [col for col in base_columns if col in df.columns]

# Check for 'peak_apex_volume'
has_apex = 'peak_apex_volume' in df.columns
if has_apex:
    cols_to_use.append('peak_apex_volume')

# Select columns and filter volume > 0
# Avoid unnecessary copy if possible, but copy is needed for filtering
df = df[cols_to_use].copy()
df = df[df['volume_ml'] > 0]

# Filter for Elute stage
elute_df = df[df[stage_col] == 'Elute'].copy()

# Find max signal per run in Elute stage to determine threshold
max_signal_per_run = elute_df.groupby('run_no')['UV_1_280_mAU'].max().reset_index()
max_signal_per_run.columns = ['run_no', 'max_run_signal']

# Merge to get max signal for each row
elute_df = elute_df.merge(max_signal_per_run, on='run_no', how='left')
```

```

# Identify the peak with the highest height (UV_1_280_mAU) in Elute stage per run
# Explicitly using 'height' (UV_1_280_mAU) as the metric
primary_peaks = elute_df.loc[elute_df.groupby('run_no')['UV_1_280_mAU'].idxmax()]

# Fallback Rule: If peak height < 10% of max run signal, flag as 'No Peak Detected'
primary_peaks['is_valid'] = primary_peaks['UV_1_280_mAU'] >= (0.10 * primary_peaks
    ['max_run_signal'])

# Separate valid and invalid runs
valid_runs = primary_peaks[primary_peaks['is_valid']].copy()
invalid_runs = primary_peaks[~primary_peaks['is_valid']].copy()

# Also check if any run has NO peaks in Elute stage at all
all_runs = df['run_no'].unique()
runs_with_elute = elute_df['run_no'].unique()
runs_missing_elute = set(all_runs) - set(runs_with_elute)

# Combine invalid runs: those with no elute peak and those with low signal
# Initialize flagged_runs as a set to avoid repeated list extensions and final set
    conversion
flagged_runs = set(runs_missing_elute)
flagged_runs.update(invalid_runs['run_no'].tolist())

# Calculate Retention Volume
# Logic to derive 'peak_apex_volume' if missing
if has_apex:
    # Ensure peak_apex_volume is numeric and > 0
    valid_runs['peak_apex_volume'] = pd.to_numeric(valid_runs['peak_apex_volume'],
        errors='coerce')
    valid_runs = valid_runs[valid_runs['peak_apex_volume'] > 0]
    retention_volumes = valid_runs['peak_apex_volume']
else:
    # Derive 'peak_apex_volume' by selecting 'volume_ml' corresponding to max '
        UV_1_280_mAU'
    # Since valid_runs already contains the row with max UV per run (from idxmax
        logic above),
    # we can directly use 'volume_ml' from these rows as the derived apex volume.
    retention_volumes = valid_runs['volume_ml']

# Calculate Statistics
total_runs_analyzed = len(valid_runs)
mean_retention_volume = retention_volumes.mean() if total_runs_analyzed > 0 else 0
std_retention_volume = retention_volumes.std() if total_runs_analyzed > 1 else 0
rsd_percent = (std_retention_volume / mean_retention_volume * 100) if
    mean_retention_volume > 0 else 0

# Flag runs exceeding USP threshold (2.0%) using Vectorized Calculation
flagged_runs_deviation = []
if total_runs_analyzed > 0:
    deviations = abs((retention_volumes - mean_retention_volume) /
        mean_retention_volume) * 100
    flagged_runs_deviation = valid_runs.loc[deviations > USP_THRESHOLD, 'run_no'].
        tolist()

# Combine all flagged runs (No Peak Detected + Deviation > 2%)
# Use set operations directly to avoid converting to list if not needed for
    iteration
all_flagged_runs = flagged_runs.union(flagged_runs_deviation)

```

```

# --- Output Generation ---

# 1. Text Report
report_lines = [
    f"Total_Runs_Analyzed:{total_runs_analyzed}",
    f"Mean_Retention_Volume:{mean_retention_volume:.4f}mL",
    f"Calculated_RSD(%):{rsd_percent:.4f}",
    f"Flagged_Runs(No_Peak_Detected_or_{>2%Deviation}):{len(all_flagged_runs)}",
    "List_of_Flagged_Runs:"
]

# Add up to 15 specific run numbers to keep within 20 lines total
for i, run in enumerate(sorted(all_flagged_runs)[:15]):
    report_lines.append(f"_{run}-_Run_{run}")

if len(all_flagged_runs) > 15:
    report_lines.append(f"_{len(all_flagged_runs)-15}_more.")

# Append to analysis_report.md as required by plan
with open(REPORT_FILE, 'a') as f:
    f.write("\n".join(report_lines) + "\n")

# 2. Visualisation
if total_runs_analyzed > 0:
    plt.figure(figsize=(10, 6))
    plt.boxplot(retention_volumes, vert=True, patch_artist=True)

    # Mean line
    plt.axvline(mean_retention_volume, color='red', linestyle='--', label=f'Mean_{mean_retention_volume:.2f}')

    # USP Limit lines (Mean +/- 2% of Mean)
    lower_limit = mean_retention_volume * (1 - USP_THRESHOLD/100)
    upper_limit = mean_retention_volume * (1 + USP_THRESHOLD/100)

    # Update plot labels to include calculated lower and upper limit values
    plt.axvline(lower_limit, color='green', linestyle='--', label=f'Lower_Limit_{lower_limit:.2f}')
    plt.axvline(upper_limit, color='green', linestyle='--', label=f'Upper_Limit_{upper_limit:.2f}')

    plt.title('Retention_Volume_Variability_and_USP_Compliance')
    plt.xlabel('Retention_Volume(mL)')
    plt.ylabel('Runs')
    plt.legend()
    plt.grid(axis='x', linestyle=':', alpha=0.6)

    plt.savefig(PLOT_FILE, dpi=150, bbox_inches='tight')
    plt.close()

print("Step3_Analysis_Complete.")

```

Listing 3: Code_3